

Лабораторная работа №1-2, Часть 1: Подготовка рабочего окружения для MLOps

Цель работы: Освоить базовые принципы установки и настройки современного рабочего окружения для Data Science и MLOps, основанного на дистрибутиве Anaconda (conda) и технологии Docker. Получить практические навыки управления виртуальными окружениями Python, работы с менеджером пакетов conda и выполнения основных операций с Docker-контейнерами.

Стек технологий:

- **Операционная система:** Ubuntu 24.04 LTS
 - **Менеджер пакетов и окружений:** Conda (в составе дистрибутива Anaconda или Miniconda)
 - **Технология контейнеризации:** Docker
 - **Платформа для интерактивных вычислений:** JupyterLab
-

Теоретическая часть (краткое содержание)

1. Введение в Conda и виртуальные окружения Современная разработка на Python, особенно в области анализа данных и машинного обучения, требует управления множеством зависимостей (библиотек) и их версий. Виртуальное окружение — это изолированное пространство, позволяющее устанавливать конкретные версии пакетов, не влияя на глобальную установку Python или другие проекты. **Conda** — это кроссплатформенный менеджер пакетов и управления окружениями с открытым исходным кодом. В отличие от стандартного **venv**, conda может управлять не только Python-пакетами, но и бинарными зависимостями (например, библиотеками C/C++), что критически важно для работы таких пакетов, как NumPy, SciPy или TensorFlow. **Anaconda** — это дистрибутив Python и R, который включает в себя conda, множество предустановленных научных пакетов (data science stack) и графические утилиты. **Miniconda** — его минималистичная версия, содержащая только conda, Python и небольшой набор базовых пакетов.

2. Введение в Docker **Docker** — это платформа для разработки, доставки и запуска приложений в контейнерах. **Контейнер** — это стандартизированная единица программного обеспечения, которая инкапсулирует код и все его зависимости, обеспечивая быстрое и надежное выполнение приложения в любой среде (ноутбук, сервер, облако). Ключевые преимущества:

- **Изоляция:** Приложение в контейнере не зависит от окружения хостовой системы.
- **Переносимость:** Образ, собранный на одной машине, гарантированно запустится на другой.
- **Повторяемость:** Исключается проблема "но на моей машине это работало".
- **Масштабируемость:** Легко запустить несколько экземпляров приложения.

Базовые концепции:

- **Docker Image:** Шаблон только для чтения, используемый для создания контейнеров. Собирается из Dockerfile.
 - **Docker Container:** Запущенный экземпляр образа.
 - **Docker Hub:** Публичный реестр для хранения и обмена образами.
-

Задание на практическую реализацию

Этап 1: Установка и настройка Miniconda

1. Загрузка и установка Miniconda:

- Откройте терминал (**Ctrl+Alt+T**).
- Загрузите последнюю версию установщика Miniconda для Linux (x86_64) с официального сайта с помощью **wget**:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

- Запустите установщик (ответьте **yes** на запрос лицензии и запуска **conda init**):

```
bash Miniconda3-latest-Linux-x86_64.sh
```

- Закройте и снова откройте терминал, чтобы активировать изменения. В начале строки приглашения терминала должно появиться **(base)**, что указывает на активацию базового окружения conda.

2. Создание и активация виртуального окружения:

- Создайте новое виртуальное окружение с именем **mlops-lab** и Python версии 3.10:

```
conda create -n mlops-lab python=3.10
```

- Активируйте созданное окружение:

```
conda activate mlops-lab
```

- Убедитесь, что приглашение терминала изменилось с **(base)** на **(mlops-lab)**.

3. Установка пакетов в созданное окружение:

- Установите основные пакеты для Data Science и последующей работы с MLflow, используя менеджер пакетов **pip**:

```
pip install numpy pandas scikit-learn matplotlib jupyterlab
```

- Проверьте установку, например, попробовав импортировать **pandas**:

```
python -c "import pandas as pd; print(pd.__version__)"
```

Этап 2: Установка Docker и базовые операции

1. Установка Docker Engine:

- Обновите индексы пакетов и установите необходимые зависимости:

```
sudo apt-get update
sudo apt-get install ca-certificates curl
```

- Добавьте официальный GPG-ключ Docker:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

- Настройте репозиторий:

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
  by=/etc/apt/keyrings/docker.asc]
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

- Установите Docker Engine:

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
```

- Проверьте установку, запустив тестовый образ:

```
sudo docker run hello-world
```

Эта команда загрузит тестовый образ и запустит его в контейнере, который выведет сообщение об успешной установке.

2. Настройка прав для не-root пользователя (ОБЯЗАТЕЛЬНО):

- По умолчанию для управления Docker требуется права `sudo`. Чтобы запускать команды `docker` без `sudo`, добавьте своего пользователя в группу `docker`:

```
sudo usermod -aG docker $USER
```

- Закройте терминал и войдите в систему заново**, чтобы изменения вступили в силу.

3. Базовые команды Docker:

- Запуск контейнера:** Запустите контейнер из образа `ubuntu:24.04` в интерактивном режиме (`-it`):

```
docker run -it ubuntu:24.04 /bin/bash
```

Вы окажетесь внутри контейнера. Попробуйте выполнить команды `ls /`, `cat /etc/os-release`. Выйдите из контейнера командой `exit`.

- Просмотр запущенных контейнеров:** Команда `docker ps` покажет работающие контейнеры. Чтобы увидеть все контейнеры (включая остановленные), используйте `docker ps -a`.
- Просмотр образов:** Команда `docker images` отобразит список образов, скачанных на ваш компьютер.
- Удаление контейнера:** Найдите ID остановленного контейнера (`docker ps -a`) и удалите его:

```
docker rm <container_id>
```

4. Запуск JupyterLab в Docker-контейнере:

- Запустите предсобранный образ JupyterLab, предоставляемый проектом Jupyter Docker Stacks. Эта команда делает порт 8888 контейнера доступным на локальной машине и монтирует текущую рабочую директорию (`$PWD`) в директорию `/home/jovyan/work` внутри контейнера:

```
docker run -p 8888:8888 -v $PWD:/home/jovyan/work jupyter/datascience-notebook:latest
```

- В логах запуска найдите URL-адрес вида `http://127.0.0.1:8888/lab?token=....`. Скопируйте его и откройте в браузере.
- Вы должны увидеть интерфейс JupyterLab. Проверьте, что в левой панели видна папка `work`, содержащая файлы из вашей текущей директории.
- Создайте новый Python-ноутбук (`File -> New -> Notebook`), выбрав ядро Python 3.
- В ячейке ноутбука выполните код для проверки окружения:

```
import numpy as np
import pandas as pd
print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
```

- Остановите контейнер, нажав **Ctrl+C** в терминале, где он запущен, или найдя его ID (`docker ps`) и выполнив `docker stop <container_id>`.

Критерии оценки для Части 1:

- **Удовлетворительно:** Успешно выполнены Этап 1 (установка Conda, создание окружения) и первые два пункта Этапа 2 (установка Docker, команда `hello-world`). Предоставлены соответствующие скриншоты.
- **Хорошо:** Дополнительно успешно выполнен пункт по запуску JupyterLab в контейнере и проверке работы ноутбука. Предоставлены скриншоты и даны ответы на контрольные вопросы.
- **Отлично:** Все задания выполнены в полном объеме. В ответах на вопросы продемонстрировано понимание принципов работы инструментов.

Рекомендуемая литература (источники)

1. **Официальная документация Conda:** <https://docs.conda.io/>
2. **Официальная документация Docker:** <https://docs.docker.com/>
3. **Jupyter Docker Stacks:** <https://jupyter-docker-stacks.readthedocs.io/>
4. **Ubuntu 24.04 Documentation:** <https://help.ubuntu.com/>
5. **Python Packaging User Guide:** <https://packaging.python.org/>
6. **Виртуальные окружения в Python:** <https://docs.python.org/3/tutorial/venv.html>
7. **Docker Get Started Tutorial:** <https://docs.docker.com/get-started/>
8. **Conda User Guide:** <https://docs.conda.io/projects/conda/en/latest/user-guide/>
9. **Статья "Зачем нужен Docker":** <https://habr.com/ru/company/ruvds/blog/438796/>
10. **Книга "Python for Data Analysis" by Wes McKinney:** (Главы о настройке окружения)

Лабораторная работа №1-2, Часть 2: Основы трекинга экспериментов с использованием MLflow

Цель работы: Освоить базовые принципы работы с платформой MLflow для управления жизненным циклом машинного обучения (MLOps). Получить практические навыки логирования параметров, метрик и артефактов вычислительного эксперимента, а также организации их хранения, визуализации и сравнения.

Стек технологий:

- **Операционная система:** Ubuntu 24.04 LTS
 - **Менеджер пакетов и окружений:** Conda (окружение `mlops-lab` из Части 1)
 - **Библиотеки:** `mlflow`, `scikit-learn`, `pandas`, `matplotlib`
 - **Платформа для трекинга:** MLflow Tracking Server (standalone)
-

Теоретическая часть (краткое содержание)

1. Введение в MLOps и MLflow MLOps (Machine Learning Operations) — это совокупность практик, направленных на автоматизацию и надежность жизненного цикла машинного обучения (развертывание, мониторинг, управление данными). Ключевая проблема, которую решает MLOps — обеспечение воспроизводимости, отслеживаемости и управляемости ML-экспериментов. **MLflow** — это open-source платформа для управления end-to-end жизненным циклом машинного обучения. Она включает в себя четыре основных компонента:

- **MLflow Tracking:** API и UI для логирования параметров, метрик, кода и артефактов (графики, модели) в процессе выполнения ML-кода.
- **MLflow Projects:** Стандартный формат упаковки ML-кода для обеспечения воспроизводимости на любой платформе.
- **MLflow Models:** Стандартный формат упаковки ML-моделей для упрощения их развертывания с помощью различных инструментов.
- **MLflow Model Registry:** Централизованное хранилище моделей для управления их жизненным циклом (staging, production, archiving).

В данной работе focuses на компоненте **Tracking**.

2. Ключевые концепции MLflow Tracking

- **Эксперимент (Experiment):** Контейнер для группы запусков (например, "Оптимизация гиперпараметров для модели X").
- **Запуск (Run):** Одно выполнение кода, которое логируется в MLflow. Каждый запуск фиксирует:
 - **Параметры (Parameters):** Входные переменные модели (например, `max_depth`, `learning_rate`).
 - **Метрики (Metrics):** Количественные показатели качества модели (например, `accuracy`, `rmse`). Метрики могут обновляться по ходу выполнения (эпохам, итерациям).
 - **Артефакты (Artifacts):** Файлы любого типа, связанные с запуском (например, графики, обученная модель, файл с предсказаниями).

- **Теги (Tags):** Произвольные ключ-значения для пометки запусков.
 - **Backend Store:** Хранилище (файловая система или база данных), где сохраняются метаданные запусков (параметры, метрики).
 - **Artifact Store:** Хранилище (например, локальная папка, S3) для артефактов.
-

Задание на практическую реализацию

Этап 1: Установка MLflow и запуск Tracking Server

1. Активация окружения и установка пакетов:

- Активируйте созданное ранее окружение `mlops-lab`:

```
conda activate mlops-lab
```

- Установите библиотеку `mlflow`:

```
pip install mlflow
```

2. Запуск MLflow Tracking Server:

- В терминале выполните команду для запуска сервера. Флаги `--backend-store-uri` и `--default-artifact-root` указывают, где хранить метаданные и артефакты соответственно (в данном случае — в локальной директории `./mlruns`).

```
mlflow server --backend-store-uri sqlite:///mlflow.db --default-artifact-root ./mlruns --host 0.0.0.0 --port 5000
```

- Сервер будет запущен и доступен в браузере по адресу `http://<your_ip_address>:5000` или `http://localhost:5000`. Вы должны увидеть интерфейс MLflow UI с списком экспериментов (пока пустой).

Этап 2: Написание и запуск скрипта с трекингом эксперимента

1. Создание Python-скрипта:

- Создайте файл `mlflow_basic.py`.
- Скопируйте в него следующий код, который реализует простое обучение модели логистической регрессии на встроенном в `sklearn` наборе данных Iris.

```
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_iris
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
import matplotlib.pyplot as plt

# Установите URI для отслеживания (указывает на запущенный сервер)
mlflow.set_tracking_uri("http://localhost:5000")

# Создайте или установите активный эксперимент
experiment_name = "Iris_Classification_Baseline"
mlflow.set_experiment(experiment_name)

# Загрузка данных
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Определите параметры модели для логирования
params = {
    "solver": "lbfgs",
    "max_iter": 1000,
    "multi_class": "auto",
    "random_state": 42
}

# Начало запуска MLflow
with mlflow.start_run():
    # Логирование параметров
    mlflow.log_params(params)

    # Создание и обучение модели
    model = LogisticRegression(**params)
    model.fit(X_train, y_train)

    # Предсказание и расчет метрик
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    # Логирование метрик
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_metric("precision", precision)
    mlflow.log_metric("recall", recall)
    mlflow.log_metric("f1_score", f1)

    # Логирование модели
    mlflow.sklearn.log_model(model, "model")

    # Создание и логирование артефакта (графика)

```

```
fig, ax = plt.subplots()
ax.bar(['Accuracy', 'Precision', 'Recall', 'F1'], [accuracy, precision,
recall, f1])
ax.set_ylabel('Score')
ax.set_title('Model Performance Metrics')
plt.savefig("metrics_plot.png") # Сохраняем график в файл
mlflow.log_artifact("metrics_plot.png") # Логируем файл как артефакт

# Вывод метрик в консоль для удобства
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")

print("Run completed and logged to MLflow!")
```

2. Запуск скрипта:

- Убедитесь, что MLflow Server запущен.
- В другом окне терминала (с активированным окружением `mlops-lab`) выполните:

```
python mlflow_basic.py
```

- В консоли должны отобразиться рассчитанные метрики и сообщение об успешном завершении.

Этап 3: Анализ результатов в MLflow UI

1. **Откройте UI:** Перейдите по адресу `http://localhost:5000`.
2. **Найдите свой эксперимент:** В боковом меню выберите эксперимент `Iris_Classification_Baseline`. Вы увидите список запусков (в данном случае один).
3. **Изучите детали запуска:**
 - Нажмите на дату/время запуска, чтобы открыть его детальную страницу.
 - Во вкладке **Overview** просмотрите логированные параметры и метрики.
 - Перейдите во вкладку **Artifacts**. Здесь вы должны увидеть:
 - Папку `model` с сохраненной моделью в формате MLflow.
 - Файл `metrics_plot.png` с построенным графиком.
4. **Сравнение запусков (опционально, для будущих работ):**
 - Запустите скрипт еще 1-2 раза, изменив какой-либо параметр (например, `"solver": "liblinear"`).
 - В UI на странице эксперимента можно будет сравнить метрики всех запусков.

Требования к оформлению и отчету (для Части 2)

Критерии оценки для Части 2:

- **Удовлетворительно:** Успешно выполнен Этап 1 (установка MLflow, запуск Tracking Server) и Этап 2 (написание и выполнение скрипта `mlflow_basic.py` без ошибок). В UI MLflow отображается один завершенный запуск.
 - **Хорошо:** Дополнительно к критерию "Удовлетворительно" в UI MLflow корректно отображаются все логированные параметры, метрики и артефакты (модель и график). Продемонстрирована работа UI.
 - **Отлично:** Все задания выполнены в полном объеме. Проведен дополнительный эксперимент: скрипт был модифицирован для логирования другой метрики (например, `log_loss`) или параметра (например, `test_size`), произведено 2 и более запусков, и в UI продемонстрировано их сравнение.
-

Рекомендуемая литература (источники)

1. **Официальная документация MLflow:** <https://mlflow.org/docs/latest/index.html>
2. **Официальный туториал MLflow Tracking:** <https://mlflow.org/docs/latest/tracking.html>
3. **Scikit-learn User Guide:** https://scikit-learn.org/stable/user_guide.html
4. **Статья "Getting Started with MLflow":** <https://towardsdatascience.com/getting-started-with-mlflow-52eff9c57d0d>
5. **Книга "Machine Learning Engineering" by Andriy Burkov:** (Главы, касающиеся MLOps и воспроизводимости экспериментов)
6. **MLflow: A Tool for Managing the Machine Learning Lifecycle:** <https://databricks.com/blog/2018/06/05/introducing-mlflow-an-open-source-machine-learning-platform.html>
7. **Документация по sklearn.metrics:** <https://scikit-learn.org/stable/modules/classes.html#module-sklearn.metrics>
8. **Matplotlib Documentation:** <https://matplotlib.org/stable/contents.html>